10. Python Program to Implement the A* Algorithm

Aim: To implement the A* search algorithm in Python to find the least-cost path from a starting node to a goal node using path cost and heuristic estimates.

Procedure:

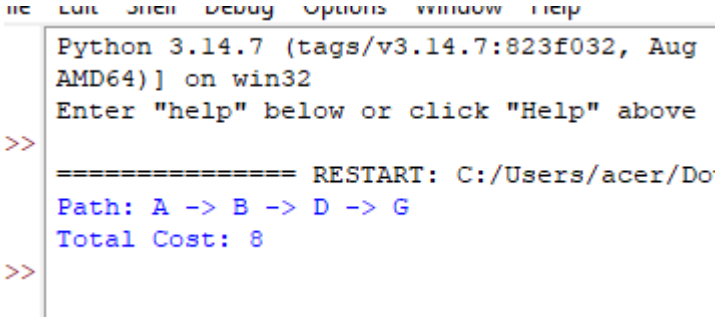
1. Define the graph with the travel costs between connected nodes.
2. Assign a heuristic value to each node representing the estimated cost to the goal.
3. Initialize the starting node with a path cost of zero.
4. Calculate $f(n) = g(n) + h(n)$, where $g(n)$is the actual path cost and $h(n)$is the heuristic estimate.
5. Expand the node with the lowest $f(n)$value and update its neighbours.
6. Continue until the goal is reached, then display the path and total cost.

```
import heapq

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 4)],
    'E': [('G', 2)],
    'F': [('G', 3)],
    'G': []
}

heuristic = {
    'A': 7, 'B': 6, 'C': 4,
    'D': 4, 'E': 2, 'F': 3, 'G': 0
}
def astar(start, goal):
    queue = [(heuristic[start], 0, start, [start])]
    best_cost = {start: 0}
    while queue:
        f, g, node, path = heapq.heappop(queue)
        if node == goal:
            return path, g
        if g > best_cost.get(node, float('inf')):
            continue
        for neighbour, weight in graph[node]:
            new_cost = g + weight
            if new_cost < best_cost.get(
                    neighbour, float('inf')):
                best_cost[neighbour] = new_cost
                new_f = new_cost + heuristic[neighbour]
                heapq.heappush(
                    queue,
                    (new_f, new_cost, neighbour,
                     path + [neighbour])
                )
    return None, float('inf')
path, cost = astar('A', 'G')
print("Path:", " -> ".join(path))
print("Total Cost:", cost)
```

Output:



Result: Thus, the A* search algorithm was implemented successfully using Python to find a least-cost path from the starting node to the goal node.